可以。下面我给你一个把这些知识点串起来的最小 demo，同时把每个知识点的基本用法也放进去。

这里用到的能力都来自 Python 标准库，例如 json、subprocess 和 venv。

这个 demo 会覆盖什么

它会把这些内容串起来：

- 变量
- if
- for
- 函数
- list
- dict
- json
- 文件读写
- subprocess
- pip + venv

一、先创建环境

```
mkdir mini_demo
cd mini_demo

python -m venv venv
```

venv 是 Python 自带的虚拟环境工具。

激活环境：

Mac / Linux

```
source venv/bin/activate
```

Windows

```
venv\Scripts\activate
```

这次 demo 不依赖第三方库，所以不用 pip install。

但以后你装包时，就是这样：

```
pip install 包名
```

二、先看完整 demo

把下面代码保存成 app.py：

```
import json
import subprocess
import os
```

```
# 1. 函数：读取文件
def read_file(path):
    with open(path, "r", encoding="utf-8") as f:
        return f.read()


# 2. 函数：写文件
def write_file(path, content):
    with open(path, "w", encoding="utf-8") as f:
        f.write(content)


# 3. 函数：执行命令
def run_command(cmd):
    result = subprocess.run(
        cmd,
        capture_output=True,
        text=True
    )
    return {
        "stdout": result.stdout,
        "stderr": result.stderr,
        "code": result.returncode
    }


# 4. 函数：处理任务
def handle_task(task):
    action = task["action"]

    if action == "write_file":
        path = task["path"]
        content = task["content"]
        write_file(path, content)
        return {"status": "success", "message": f"{path} 写入
成功"}

    elif action == "read_file":
        path = task["path"]
        if not os.path.exists(path):
            return {"status": "error", "message": f"{path} 不
存在"}
        content = read_file(path)
        return {"status": "success", "content": content}
```

```
    elif action == "run_command":
        cmd = task["cmd"]
        result = run_command(cmd)
        return {"status": "success", "result": result}

    else:
        return {"status": "error", "message": "未知操作"}


# 5. 主流程
def main():
    # list：一组任务
    task_list = [
        {
            "action": "write_file",
            "path": "hello.txt",
            "content": "Hello, mini Claude Code!"
        },
        {
            "action": "read_file",
            "path": "hello.txt"
        },
        {
            "action": "run_command",
            "cmd": ["python", "--version"]
        }
    ]

    # for：遍历任务
    for task in task_list:
        print("\n当前任务：", task)

        # dict -> JSON 字符串
        task_json = json.dumps(task, ensure_ascii=False)
        print("转成 JSON：", task_json)

        # JSON 字符串 -> dict
        parsed_task = json.loads(task_json)

        try:
            result = handle_task(parsed_task)
            print("执行结果：", result)
        except Exception as e:
```

```
            print("出错了：", str(e))


if __name__ == "__main__":
    main()
```

运行：

```
python app.py
```

三、这个 demo 里每个知识点怎么用

1. 变量

```
action = task["action"]
path = task["path"]
content = task["content"]
```

就是拿一个名字去存数据。

你以后最常见的就是：

- 存文件路径
- 存文件内容
- 存命令输出
- 存 AI 返回结果

2. if

```
if action == "write_file":
    ...
elif action == "read_file":
    ...
else:
    ...
```

作用就是：

根据不同条件，走不同分支。

你以后会经常拿它做：

- 判断 AI 想调用哪个工具
- 判断命令成功还是失败
- 判断文件是否存在

3. for

```
for task in task_list:
    ...
```

作用就是：

把列表里的东西一个一个处理。

你以后会拿它处理：

- 多个文件
- 多个任务
- 多条消息

4. 函数

```
def read_file(path):
    ...
```

函数就是把一段逻辑包装起来，以后重复用。

你以后项目里最常见的函数就是这类：

- read_file
- write_file
- run_command
- call_llm

5. list

```
task_list = [
    {...},
    {...},
    {...}
]
```

列表就是一组有顺序的数据。

常见操作：

```
files = ["a.py", "b.py"]
files.append("c.py")
print(files[0])
```

你以后会用 list 存：

- 文件列表
- 任务列表
- 命令参数

6. dict

```
task = {
    "action": "read_file",
    "path": "hello.txt"
}
```

字典就是“键值对”。

取值：

```
task["action"]
task["path"]
```

这个最重要，因为 agent 系统里最常见的数据结构就是 dict。

7. JSON

这个 demo 里有两步：

dict 转 JSON

```
task_json = json.dumps(task, ensure_ascii=False)
```

JSON 转 dict

```
parsed_task = json.loads(task_json)
```

你以后会经常遇到这种情况：

- 模型输出一段 JSON
- 你把它解析成 dict
- 再去调用工具

8. 文件读写

写文件

```
write_file(path, content)
```

本质是：

```
with open(path, "w", encoding="utf-8") as f:
    f.write(content)
```

读文件

```
content = read_file(path)
```

本质是：

```
with open(path, "r", encoding="utf-8") as f:
    return f.read()
```

你以后做 Claude Code 类工具，文件读写是核心动作。

9. subprocess

这个 demo 里跑的是：

```
["python", "--version"]
```

代码是：

```
result = subprocess.run(
    cmd,
    capture_output=True,
    text=True
)
```

然后取：

```
result.stdout
result.stderr
result.returncode
```

这三个特别重要：

- stdout：正常输出
- stderr：错误输出
- returncode：退出码，0 通常表示成功

你以后会拿它跑：

- pytest
- git status
- npm test
- mvn test

10. try / except

```
try:
    result = handle_task(parsed_task)
except Exception as e:
    print("出错了：", str(e))
```

作用就是：

防止程序一出错就直接崩掉。

这在 agent 项目里非常重要，因为：

- 文件可能不存在
- 命令可能失败
- JSON 可能格式不对

四、你可以立刻改着玩的地方

改 1：增加一个新任务

在 task_list 里加：

```
{
    "action": "write_file",
    "path": "note.txt",
    "content": "这是第二个文件"
}
```

改 2：把命令换成别的

比如：

```
{
    "action": "run_command",
    "cmd": ["git", "status"]
}
```

或者：

```
{
    "action": "run_command",
    "cmd": ["ls"]
}
```

Windows 可以换成：

```
{
    "action": "run_command",
    "cmd": ["cmd", "/c", "dir"]
}
```

改 3：故意让它报错

比如读一个不存在的文件：

```
{
    "action": "read_file",
    "path": "not_exists.txt"
}
```

这样你就能看到错误处理是怎么工作的。

五、这个 demo 本质上是什么

这已经是一个最小 Agent 执行器了：

1. 用 dict 表示任务
2. 用 json 模拟 AI 输出
3. 用 if 决定调用哪个工具
4. 用文件读写和 subprocess 真正执行动作
5. 用 try/except 保证程序不崩

也就是说，它已经有了“简化版 Claude Code”的雏形。

六、你现在最该记住的基本用法

直接记这几句就够了：

```
# 读 JSON
data = json.loads(s)

# 写 JSON
s = json.dumps(data, ensure_ascii=False)

# 读文件
content = open("a.txt", "r", encoding="utf-8").read()

# 写文件
open("a.txt", "w", encoding="utf-8").write("hello")

# 跑命令
result = subprocess.run(["python", "--version"],
capture_output=True, text=True)

# 判断成功失败
if result.returncode == 0:
    print(result.stdout)
else:
    print(result.stderr)
```

七、你下一步最适合做什么

下一步不要继续学零散语法了。

最适合的是把这个 demo 升级成：

- 用户输入一句话
- 程序生成一个任务 dict
- 自动执行
- 输出结果

这样你就正式开始做 mini Claude Code 了。

如果你愿意，我下一条直接给你一个**“带用户输入的升级版 demo”**。